

CrewAI: Introduction

What is CrewAI?

CrewAI is the leading open-source framework for orchestrating autonomous AI agents and building complex workflows.

It empowers developers to build production-ready multi-agent systems by combining the collaborative intelligence of **Crews** with the precise control of **Flows**.

- **CrewAI Flows:** The backbone of your AI application. Flows allow you to create structured, event-driven workflows that manage state and control execution. They provide the scaffolding for your AI agents to work within.
- **CrewAI Crews:** The units of work within your Flow. Crews are teams of autonomous agents that collaborate to solve specific tasks delegated to them by the Flow.

With over 100,000 developers certified through our community courses, CrewAI is the standard for enterprise-ready AI automation.

Watch: Building CrewAI Agents & Flows with Coding Agent Skills

Install our coding agent skills (Claude Code, Codex, ...) to quickly get your coding agents up and running with CrewAI.

You can install it with `npx skills add crewaiinc/skills`

The CrewAI Architecture

CrewAI's architecture is designed to balance autonomy with control.

1. Flows: The Backbone

Think of a Flow as the "manager" or the "process definition" of your application. It defines the steps, the logic, and how data moves through your system.

Flows provide:

- **State Management:** Persist data across steps and executions.
- **Event-Driven Execution:** Trigger actions based on events or external inputs.
- **Control Flow:** Use conditional logic, loops, and branching.

2. Crews: The Intelligence

Crews are the "teams" that do the heavy lifting. Within a Flow, you can trigger a Crew to tackle a complex problem requiring creativity and collaboration.

Crews provide:

- **Role-Playing Agents:** Specialized agents with specific goals and tools.
- **Autonomous Collaboration:** Agents work together to solve tasks.
- **Task Delegation:** Tasks are assigned and executed based on agent capabilities.

How It All Works Together

1. **The Flow** triggers an event or starts a process.
2. **The Flow** manages the state and decides what to do next.

3. **The Flow** delegates a complex task to a **Crew**.
4. **The Crew's** agents collaborate to complete the task.
5. **The Crew** returns the result to the **Flow**.
6. **The Flow** continues execution based on the result.

Key Features

Build reliable, stateful workflows that can handle long-running processes and complex logic.

Deploy teams of agents that can plan, execute, and collaborate to achieve high-level goals.

Connect your agents to any API, database, or local tool.

Designed with security and compliance in mind for enterprise deployments.

When to Use Crews vs. Flows

The short answer: Use both.

For any production-ready application, **start with a Flow**.

- **Use a Flow** to define the overall structure, state, and logic of your application.
- **Use a Crew** within a Flow step when you need a team of agents to perform a specific, complex task that requires autonomy.

Use Case - Architecture

Simple Automation - Single Flow with Python tasks

Complex Research - Flow managing state -> Crew performing research

Application Backend - Flow handling API requests -> Crew generating content -> Flow saving to DB

Why Choose CrewAI?

- **Autonomous Operation:** Agents make intelligent decisions based on their roles and available tools
- **Natural Interaction:** Agents communicate and collaborate like human team members
- **Extensible Design:** Easy to add new tools, roles, and capabilities
- **Production Ready:** Built for reliability and scalability in real-world applications
- **Security-Focused:** Designed with enterprise security requirements in mind
- **Cost-Efficient:** Optimized to minimize token usage and API calls

Ready to Start Building?

Learn how to create structured, event-driven workflows with precise control over execution.

Step-by-step tutorial to create a collaborative AI team that works together to solve complex problems.

Get started with CrewAI in your development environment.

Scaffold a Flow, run a crew with one agent, and generate a report end to end.

Connect with other developers, get help, and share your CrewAI experiences.

CrewAI Concepts: Agents

Overview of an Agent

In the CrewAI framework, an Agent is an autonomous unit that can:

- Perform specific tasks
- Make decisions based on its role and goal
- Use tools to accomplish objectives
- Communicate and collaborate with other agents
- Maintain memory of interactions
- Delegate tasks when allowed

Think of an agent as a specialized team member with specific skills, expertise, and responsibilities. For example, a `Researcher` agent might excel at gathering and analyzing information, while a `Writer` agent might be better at creating content.

CrewAI AMP includes a Visual Agent Builder that simplifies agent creation and configuration without writing code. Design your agents visually and test them in real-time.

!Visual Agent Builder Screenshot

The Visual Agent Builder enables:

- Intuitive agent configuration with form-based interfaces
- Real-time testing and validation
- Template library with pre-configured agent types
- Easy customization of agent attributes and behaviors

Agent Attributes

Attribute - Parameter - Type - Description

Role - `role` - `str` - Defines the agent's function and expertise within the crew.

Goal - `goal` - `str` - The individual objective that guides the agent's decision-making.

Backstory - `backstory` - `str` - Provides context and personality to the agent, enriching interactions.

LLM `_(optional)_` - `llm` - `Union[str, LLM, Any]` - Language model that powers the agent. Defaults to the model specified in `OPENAI_MODEL_NAME` or "gpt-4".

Tools `_(optional)_` - `tools` - `List[BaseTool]` - Capabilities or functions available to the agent. Defaults to an empty list.

Function Calling LLM `_(optional)_` - `function_calling_llm` - `Optional[Any]` - Language model for tool calling, overrides crew's LLM if specified.

Max Iterations `_(optional)_` - `max_iter` - `int` - Maximum iterations before the agent must provide its best answer. Default is 20.

Max RPM `_(optional)_` - `max_rpm` - `Optional[int]` - Maximum requests per minute to avoid rate limits.

Max Execution Time `_(optional)_` - `max_execution_time` - `Optional[int]` - Maximum time (in seconds) for task execution.

Verbose `_(optional)_` - `verbose` - `bool` - Enable detailed execution logs for debugging. Default is `False`.

Allow Delegation `_(optional)_` - `allow_delegation` - `bool` - Allow the agent to delegate tasks to other agents. Default is `False`.

Step Callback `_(optional)_` - `step_callback` - `Optional[Any]` - Function called after each agent step, overrides crew callback.

Cache `_(optional)_` - `cache` - `bool` - Enable caching for tool usage. Default is `True`.

System Template `_(optional)_` - `system_template` - `Optional[str]` - Custom system prompt template for the agent.

Prompt Template `_(optional)_` - `prompt_template` - `Optional[str]` - Custom prompt template for the agent.

Response Template `_(optional)_` - `response_template` - `Optional[str]` - Custom response template for the agent.

Allow Code Execution `_(optional)_` - `allow_code_execution` - `Optional[bool]` - Enable code execution for the agent. Default is `False`.

Max Retry Limit `_(optional)_` - `max_retry_limit` - `int` - Maximum number of retries when an error occurs. Default is `2`.

Respect Context Window `_(optional)_` - `respect_context_window` - `bool` - Keep messages under context window size by summarizing. Default is `True`.

Code Execution Mode `_(optional)_` - `code_execution_mode` - `Literal["safe", "unsafe"]` - Mode for code execution: 'safe' (using Docker) or 'unsafe' (direct). Default is 'safe'.

Multimodal `_(optional)_` - `multimodal` - `bool` - Whether the agent supports multimodal capabilities. Default is `False`.

Inject Date `_(optional)_` - `inject_date` - `bool` - Whether to automatically inject the current date into the agent's prompt. Default is `False`.

Date Format `_(optional)_` - `date_format` - `str` - Format string for date when `inject_date` is enabled. Default is `"%Y-%m-%d"` (ISO format).

Reasoning `_(optional)_` - `reasoning` - `bool` - Whether the agent should reflect and create a plan before executing a task. Default is `False`.

Max Reasoning Attempts `_(optional)_` - `max_reasoning_attempts` - `Optional[int]` - Maximum number of reasoning attempts before executing the task. If `None`, will try until ready.

Embedder `_(optional)_` - `embedder` - `Optional[Dict[str, Any]]` - Configuration for the embedder used by the agent.

Knowledge Sources `_(optional)_` - `knowledge_sources` - `Optional[List[BaseKnowledgeSource]]` - Knowledge sources available to the agent.

Use System Prompt `_(optional)_` - `use_system_prompt` - `Optional[bool]` - Whether to use system prompt (for o1 model support). Default is `True`.

Creating Agents

There are two common ways to create agents in CrewAI: using **JSONC project configuration (recommended for new crews)** or defining them **directly in code**.

JSONC Configuration (Recommended)

New projects created with `crewai create crew` use JSON-first configuration. Each agent is defined in `agents/.jsonc`, and `crew.jsonc` lists which agents are part of the crew.

After creating your CrewAI project as outlined in the Installation section, edit the generated files in `agents/`.

Use `{placeholder}` values in `role`, `goal`, or `backstory`. Put defaults in `crew.jsonc` under `inputs`; `crewai run` prompts for any missing values.

Here's an example `agents/researcher.jsonc` file:

```
{
  "role": "{topic} Senior Data Researcher",
  "goal": "Uncover cutting-edge developments in {topic}",
  "backstory": "You find the most relevant information and present it clearly.",
  "llm": "openai/gpt-4o",
  "tools": ["SerperDevTool"],
  "settings": {
    "verbose": true,
    "allow_delegation": false,
    "max_iter": 20
  }
}
```

Then include that agent from `crew.jsonc`:

```
{
  "name": "Research Crew",
  "agents": ["researcher"],
  "tasks": [
    {
      "name": "research_task",
      "description": "Research {topic}",
      "expected_output": "A concise briefing about {topic}",
      "agent": "researcher"
    }
  ],
  "inputs": {
    "topic": "AI Agents"
  }
}
```

Agent files support any public Agent field. Common fields include `role`, `goal`, `backstory`, `llm`, `tools`, `function_calling_llm`, `guardrail`, `step_callback`, and `settings`. Behavior options such as `verbose`, `allow_delegation`, `max_iter`, `max_rpm`, `memory`, `cache`, `planning_config`, and `use_system_prompt` can be placed at the top level or under `settings`; values in `settings` take precedence.

JSONC supports comments and trailing commas. If both `agents/.jsonc` and `agents/.json` exist, CrewAI uses the JSONC file.

Classic YAML Configuration

Classic projects created with `crewai create crew --classic` use `config/agents.yaml` and a `@CrewBase` class in `crew.py`. This remains supported for teams that want Python decorators or existing YAML projects.

Direct Code Definition

You can create agents directly in code by instantiating the Agent class. Here's a comprehensive example showing all available parameters:

```
from crewai import Agent
from crewai_tools import SerperDevTool

# Create an agent with all available parameters
agent = Agent(
    role="Senior Data Scientist",
    goal="Analyze and interpret complex datasets to provide actionable insights",
```

```

backstory="With over 10 years of experience in data science and machine learning, "
        "you excel at finding patterns in complex datasets.",
llm="gpt-4", # Default: OPENAI_MODEL_NAME or "gpt-4"
function_calling_llm=None, # Optional: Separate LLM for tool calling
verbose=False, # Default: False
allow_delegation=False, # Default: False
max_iter=20, # Default: 20 iterations
max_rpm=None, # Optional: Rate limit for API calls
max_execution_time=None, # Optional: Maximum execution time in seconds
max_retry_limit=2, # Default: 2 retries on error
allow_code_execution=False, # Default: False
code_execution_mode="safe", # Default: "safe" (options: "safe", "unsafe")
respect_context_window=True, # Default: True
use_system_prompt=True, # Default: True
multimodal=False, # Default: False
inject_date=False, # Default: False
date_format="%Y-%m-%d", # Default: ISO format
reasoning=False, # Default: False
max_reasoning_attempts=None, # Default: None
tools=[SerperDevTool()], # Optional: List of tools
knowledge_sources=None, # Optional: List of knowledge sources
embedder=None, # Optional: Custom embedder configuration
system_template=None, # Optional: Custom system prompt template
prompt_template=None, # Optional: Custom prompt template
response_template=None, # Optional: Custom response template
step_callback=None, # Optional: Callback function for monitoring
)

```

Let's break down some key parameter combinations for common use cases:

Basic Research Agent

```

research_agent = Agent(
    role="Research Analyst",
    goal="Find and summarize information about specific topics",
    backstory="You are an experienced researcher with attention to detail",
    tools=[SerperDevTool()],
    verbose=True # Enable logging for debugging
)

```

Code Development Agent

```

dev_agent = Agent(
    role="Senior Python Developer",
    goal="Write and debug Python code",
    backstory="Expert Python developer with 10 years of experience",
    allow_code_execution=True,
    code_execution_mode="safe", # Uses Docker for safety
    max_execution_time=300, # 5-minute timeout
    max_retry_limit=3 # More retries for complex code tasks
)

```

Long-Running Analysis Agent

```

analysis_agent = Agent(
    role="Data Analyst",
    goal="Perform deep analysis of large datasets",
    backstory="Specialized in big data analysis and pattern recognition",
    memory=True,
    respect_context_window=True,
    max_rpm=10, # Limit API calls
    function_calling_llm="gpt-4o-mini" # Cheaper model for tool calls
)

```

Custom Template Agent

```

custom_agent = Agent(
    role="Customer Service Representative",
    goal="Assist customers with their inquiries",
    backstory="Experienced in customer support with a focus on satisfaction",
    system_template="""<|start_header_id|>system<|end_header_id|>
        {{ .System }}<|eot_id|>""",
    prompt_template="""<|start_header_id|>user<|end_header_id|>
        {{ .Prompt }}<|eot_id|>""",
    response_template="""<|start_header_id|>assistant<|end_header_id|>
        {{ .Response }}<|eot_id|>""",
)

```

)

Date-Aware Agent with Reasoning

```
strategic_agent = Agent(
    role="Market Analyst",
    goal="Track market movements with precise date references and strategic planning",
    backstory="Expert in time-sensitive financial analysis and strategic reporting",
    inject_date=True, # Automatically inject current date into the prompt
    date_format="%B %d, %Y", # Format as "May 21, 2025"
    reasoning=True, # Enable strategic planning
    max_reasoning_attempts=2, # Limit planning iterations
    verbose=True
)
```

Reasoning Agent

```
reasoning_agent = Agent(
    role="Strategic Planner",
    goal="Analyze complex problems and create detailed execution plans",
    backstory="Expert strategic planner who methodically breaks down complex challenges",
    reasoning=True, # Enable reasoning and planning
    max_reasoning_attempts=3, # Limit reasoning attempts
    max_iter=30, # Allow more iterations for complex planning
    verbose=True
)
```

Multimodal Agent

```
multimodal_agent = Agent(
    role="Visual Content Analyst",
    goal="Analyze and process both text and visual content",
    backstory="Specialized in multimodal analysis combining text and image understanding",
    multimodal=True, # Enable multimodal capabilities
    verbose=True
)
```

Parameter Details

Critical Parameters

- role, goal, and backstory are required and shape the agent's behavior
- llm determines the language model used (default: OpenAI's GPT-4)

Memory and Context

- memory: Enable to maintain conversation history
- respect_context_window: Prevents token limit issues
- knowledge_sources: Add domain-specific knowledge bases

Execution Control

- max_iter: Maximum attempts before giving best answer
- max_execution_time: Timeout in seconds
- max_rpm: Rate limiting for API calls
- max_retry_limit: Retries on error

Code Execution

allow_code_execution and code_execution_mode are deprecated. CodeInterpreterTool has been removed from crewai-tools. Use a dedicated sandbox service such as E2B or Modal for secure code execution.

- allow_code_execution_(deprecated): Previously enabled built-in code execution via CodeInterpreterTool.

- `code_execution_mode_(deprecated)_`: Previously controlled execution mode ("safe" for Docker, "unsafe" for direct execution).

Advanced Features

- `multimodal`: Enable multimodal capabilities for processing text and visual content
- `reasoning`: Enable agent to reflect and create plans before executing tasks
- `inject_date`: Automatically inject current date into the agents prompt

Templates

- `system_template`: Defines agent's core behavior
- `prompt_template`: Structures input format
- `response_template`: Formats agent responses

When using custom templates, ensure that both `system_template` and `prompt_template` are defined. The `response_template` is optional but recommended for consistent output formatting.

When using custom templates, you can use variables like `{role}`, `{goal}`, and `{backstory}` in your templates. These will be automatically populated during execution.

Agent Tools

Agents can be equipped with various tools to enhance their capabilities. CrewAI supports tools from:

- CrewAI Toolkit
- LangChain Tools

Here's how to add tools to an agent:

```
from crewai import Agent
from crewai_tools import SerperDevTool, WikipediaTools

# Create tools
search_tool = SerperDevTool()
wiki_tool = WikipediaTools()

# Add tools to agent
researcher = Agent(
    role="AI Technology Researcher",
    goal="Research the latest AI developments",
    tools=[search_tool, wiki_tool],
    verbose=True
)
```

Agent Memory and Context

Agents can maintain memory of their interactions and use context from previous tasks. This is particularly useful for complex workflows where information needs to be retained across multiple tasks.

```
from crewai import Agent

analyst = Agent(
    role="Data Analyst",
    goal="Analyze and remember complex data patterns",
    memory=True, # Enable memory
    verbose=True
)
```

When memory is enabled, the agent will maintain context across multiple interactions, improving its ability to handle complex, multi-step tasks.

Context Window Management

CrewAI includes sophisticated automatic context window management to handle situations where conversations exceed the language model's token limits. This powerful feature is controlled by the `respect_context_window` parameter.

How Context Window Management Works

When an agent's conversation history grows too large for the LLM's context window, CrewAI automatically detects this situation and can either:

1. **Automatically summarize content** (when `respect_context_window=True`)
2. **Stop execution with an error** (when `respect_context_window=False`)

Automatic Context Handling (respect_context_window=True)

This is the **default and recommended setting** for most use cases. When enabled, CrewAI will:

```
# Agent with automatic context management (default)
smart_agent = Agent(
    role="Research Analyst",
    goal="Analyze large documents and datasets",
    backstory="Expert at processing extensive information",
    respect_context_window=True, # Default: auto-handle context limits
    verbose=True
)
```

What happens when context limits are exceeded:

- **Warning message:** "Context length exceeded. Summarizing content to fit the model context window."
- **Automatic summarization:** CrewAI intelligently summarizes the conversation history
- **Continued execution:** Task execution continues seamlessly with the summarized context
- **Preserved information:** Key information is retained while reducing token count

Strict Context Limits (respect_context_window=False)

When you need precise control and prefer execution to stop rather than lose any information:

```
# Agent with strict context limits
strict_agent = Agent(
    role="Legal Document Reviewer",
    goal="Provide precise legal analysis without information loss",
    backstory="Legal expert requiring complete context for accurate analysis",
    respect_context_window=False, # Stop execution on context limit
    verbose=True
)
```

What happens when context limits are exceeded:

- **Error message:** "Context length exceeded. Consider using smaller text or RAG tools from `crewai_tools`."
- **Execution stops:** Task execution halts immediately
- **Manual intervention required:** You need to modify your approach

Choosing the Right Setting

Use `respect_context_window=True` (Default) when:

- **Processing large documents** that might exceed context limits
- **Long-running conversations** where some summarization is acceptable
- **Research tasks** where general context is more important than exact details
- **Prototyping and development** where you want robust execution

```
# Perfect for document processing
document_processor = Agent(
    role="Document Analyst",
    goal="Extract insights from large research papers",
    backstory="Expert at analyzing extensive documentation",
    respect_context_window=True, # Handle large documents gracefully
    max_iter=50, # Allow more iterations for complex analysis
    verbose=True
)
```

Use `respect_context_window=False` when:

- **Precision is critical** and information loss is unacceptable
- **Legal or medical tasks** requiring complete context
- **Code review** where missing details could introduce bugs
- **Financial analysis** where accuracy is paramount

```
# Perfect for precision tasks
precision_agent = Agent(
    role="Code Security Auditor",
    goal="Identify security vulnerabilities in code",
    backstory="Security expert requiring complete code context",
    respect_context_window=False, # Prefer failure over incomplete analysis
    max_retry_limit=1, # Fail fast on context issues
    verbose=True
)
```

Alternative Approaches for Large Data

When dealing with very large datasets, consider these strategies:

1. Use RAG Tools

```
from crewai_tools import RagTool

# Create RAG tool for large document processing
rag_tool = RagTool()

rag_agent = Agent(
    role="Research Assistant",
    goal="Query large knowledge bases efficiently",
    backstory="Expert at using RAG tools for information retrieval",
    tools=[rag_tool], # Use RAG instead of large context windows
    respect_context_window=True,
    verbose=True
)
```

2. Use Knowledge Sources

```
# Use knowledge sources instead of large prompts
knowledge_agent = Agent(
    role="Knowledge Expert",
    goal="Answer questions using curated knowledge",
    backstory="Expert at leveraging structured knowledge sources",
    knowledge_sources=[your_knowledge_sources], # Pre-processed knowledge
    respect_context_window=True,
    verbose=True
)
```

Context Window Best Practices

1. **Monitor Context Usage:** Enable `verbose=True` to see context management in action
2. **Design for Efficiency:** Structure tasks to minimize context accumulation
3. **Use Appropriate Models:** Choose LLMs with context windows suitable for your tasks
4. **Test Both Settings:** Try both `True` and `False` to see which works better for your use case
5. **Combine with RAG:** Use RAG tools for very large datasets instead of relying solely on context windows

Troubleshooting Context Issues

If you're getting context limit errors:

```
# Quick fix: Enable automatic handling
agent.respect_context_window = True

# Better solution: Use RAG tools for large data
from crewai_tools import RagTool
agent.tools = [RagTool()]

# Alternative: Break tasks into smaller pieces
# Or use knowledge sources instead of large prompts
```

If automatic summarization loses important information:

```
# Disable auto-summarization and use RAG instead
agent = Agent(
    role="Detailed Analyst",
    goal="Maintain complete information accuracy",
    backstory="Expert requiring full context",
    respect_context_window=False, # No summarization
    tools=[RagTool()], # Use RAG for large data
    verbose=True
)
```

The context window management feature works automatically in the background. You don't need to call any special functions - just set `respect_context_window` to your preferred behavior and CrewAI handles the rest!

Direct Agent Interaction with `kickoff()`

Agents can be used directly without going through a task or crew workflow using the `kickoff()` method. This provides a simpler way to interact with an agent when you don't need the full crew orchestration capabilities.

How `kickoff()` Works

The `kickoff()` method allows you to send messages directly to an agent and get a response, similar to how you would interact with an LLM but with all the agent's capabilities (tools, reasoning, etc.).

```
from crewai import Agent
from crewai_tools import SerperDevTool

# Create an agent
researcher = Agent(
    role="AI Technology Researcher",
    goal="Research the latest AI developments",
    tools=[SerperDevTool()],
    verbose=True
)

# Use kickoff() to interact directly with the agent
result = researcher.kickoff("What are the latest developments in language models?")

# Access the raw response
print(result.raw)
```

Parameters and Return Values

Parameter - Type - Description

`messages` - Union[str, List[Dict[str, str]]] - Either a string query or a list of message dictionaries with role/content

`response_format` - Optional[Type[Any]] - Optional Pydantic model for structured output

The method returns a `LiteAgentOutput` object with the following properties:

- `raw`: String containing the raw output text

- `pydantic`: Parsed Pydantic model (if a `response_format` was provided)
- `agent_role`: Role of the agent that produced the output
- `usage_metrics`: Token usage metrics for the execution

Structured Output

You can get structured output by providing a Pydantic model as the `response_format`:

```
from pydantic import BaseModel
from typing import List

class ResearchFindings(BaseModel):
    main_points: List[str]
    key_technologies: List[str]
    future_predictions: str

# Get structured output
result = researcher.kickoff(
    "Summarize the latest developments in AI for 2025",
    response_format=ResearchFindings
)

# Access structured data
print(result.pydantic.main_points)
print(result.pydantic.future_predictions)
```

Multiple Messages

You can also provide a conversation history as a list of message dictionaries:

```
messages = [
    {"role": "user", "content": "I need information about large language models"},
    {"role": "assistant", "content": "I'd be happy to help with that! What specifically would you like to know"},
    {"role": "user", "content": "What are the latest developments in 2025?"}
]

result = researcher.kickoff(messages)
```

The last user message is the request the agent answers. Every other message keeps its role and its position around that request, so a conversation that ends in assistant or tool messages still asks the user's question and still delivers those trailing turns after it. With no user message at all, the last message is treated as the request.

Async Support

An asynchronous version is available via `kickoff_async()` with the same parameters:

```
import asyncio

async def main():
    result = await researcher.kickoff_async("What are the latest developments in AI?")
    print(result.raw)

asyncio.run(main())
```

The `kickoff()` method uses a `LiteAgent` internally, which provides a simpler execution flow while preserving all of the agent's configuration (role, goal, backstory, tools, etc.).

Important Considerations and Best Practices

Security and Code Execution

`allow_code_execution` and `code_execution_mode` are deprecated and `CodeInterpreterTool` has been removed. Use a dedicated sandbox service such as E2B or Modal for secure code execution.

Performance Optimization

- Use `respect_context_window: true` to prevent token limit issues

- Set appropriate `max_rpm` to avoid rate limiting
- Enable cache: `true` to improve performance for repetitive tasks
- Adjust `max_iter` and `max_retry_limit` based on task complexity

Memory and Context Management

- Leverage `knowledge_sources` for domain-specific information
- Configure `embedder` when using custom embedding models
- Use custom templates (`system_template`, `prompt_template`, `response_template`) for fine-grained control over agent behavior

Advanced Features

- Enable reasoning: `true` for agents that need to plan and reflect before executing complex tasks
- Set appropriate `max_reasoning_attempts` to control planning iterations (None for unlimited attempts)
- Use `inject_date`: `true` to provide agents with current date awareness for time-sensitive tasks
- Customize the date format with `date_format` using standard Python datetime format codes
- Enable `multimodal`: `true` for agents that need to process both text and visual content

Agent Collaboration

- Enable `allow_delegation`: `true` when agents need to work together
- Use `step_callback` to monitor and log agent interactions
- Consider using different LLMs for different purposes:
 - Main `llm` for complex reasoning
 - `function_calling_llm` for efficient tool usage

Date Awareness and Reasoning

- Use `inject_date`: `true` to provide agents with current date awareness for time-sensitive tasks
- Customize the date format with `date_format` using standard Python datetime format codes
- Valid format codes include: `%Y` (year), `%m` (month), `%d` (day), `%B` (full month name), etc.
- Invalid date formats will be logged as warnings and will not modify the task description
- Enable reasoning: `true` for complex tasks that benefit from upfront planning and reflection

Model Compatibility

- Set `use_system_prompt`: `false` for older models that don't support system messages
- Ensure your chosen `llm` supports the features you need (like function calling)

Troubleshooting Common Issues

1. **Rate Limiting:** If you're hitting API rate limits:
 - Implement appropriate `max_rpm`
 - Use caching for repetitive operations
 - Consider batching requests
2. **Context Window Errors:** If you're exceeding context limits:

- Enable `respect_context_window`
- Use more efficient prompts
- Clear agent memory periodically

3. **Code Execution Issues:** If code execution fails:

- Verify Docker is installed for safe mode
- Check execution permissions
- Review code sandbox settings

4. **Memory Issues:** If agent responses seem inconsistent:

- Check knowledge source configuration
- Review conversation history management

Remember that agents are most effective when configured according to their specific use case. Take time to understand your requirements and adjust these parameters accordingly.